

P2993R0

Proposal to allow simd overloads for standard C++ `<bit>` header

Published Proposal, 2023-08-01

This version:

<http://wg21.link/P2933R0>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Proposal to extend `std::simd` with overloads from `<bit>` library.

Table of Contents

1 Motivation

2 Support for `<bit>`

3 Wording

3.1 Add new section [`simd.bit`]

References

Informative References

§ 1. Motivation

[P1928R4] introduced data parallel types to C++. It mostly provided operators which worked on or with `std::simd` types, but it also included overloads of useful functions from other parts of C++ (e.g., `sin`, `cos`, `abs`). In this paper we propose some other functions from the standard C++ `<bit>` header which should receive overloads to work with `std::simd` types.

§ 2. Support for `<bit>`

The `<bit>` header is part of the numerics library and provides utilities for manipulating and querying the properties of integral values when treated as collections of bits. The table below summarises the contents of `<bit>`.

Name	Purpose	Proposed (Y/N)
<code>endian</code>	A type which indicates the endianness of scalar types.	N
<code>bit_cast</code>	reinterpret the object representation of one type as that of another	N
<code>byteswap</code>	reverses the bytes in the given integer value	Y
<code>has_single_bit</code>	checks if a number is an integral power of two	Y
<code>bit_ceil</code>	finds the smallest integral power of two not less than the given value	Y
<code>bit_floor</code>	finds the largest integral power of two not greater than the given value	Y
<code>bit_width</code>	finds the smallest number of bits needed to represent the given value	Y
<code>rotr</code>	computes the result of bitwise left-rotation	Y
<code>rotr</code>	computes the result of bitwise right-rotation	Y
<code>countl_zero</code>	counts the number of consecutive 0 bits, starting from the most significant bit	Y
<code>countl_one</code>	counts the number of consecutive 1 bits, starting from the most significant bit	Y
<code>countr_zero</code>	counts the number of consecutive 0 bits, starting from the least significant bit	Y
<code>countr_one</code>	counts the number of consecutive 1 bits, starting from the least significant bit	Y
<code>popcount</code>	counts the number of 1 bits in an unsigned integer	Y

Of these types and functions, only the first two shouldn't be handled by `std::simd`:

- `endian` indicates the endianness of a scalar type. A simd value with elements of the underlying scalar type will have the same properties and does not need special handling in `std::simd`.
- `bit_cast` should be handled differently for `std::simd` values, and a separate proposal for `simd_bit_cast` will be provided.

All the other functions from `<bit>` should be handled in `std::simd` by element-wise application of the function to each element of the SIMD value. Any constraints and behaviours on the function will be applied at the SIMD value level (e.g., `byteswap` only participates if the type is `std::integral`, so the SIMD variant of `byteswap` should also apply if its `value_type` also satisfies `std::integral`).

One small modification to the behaviour of `<bit>` for simd is where the return type differs to the input type. For example, the standard `<bit>` header defines some query functions as returning integer values:

```
template< class T >
constexpr int bit_width( T x ) noexcept;

template< class T >
constexpr int countl_one( T x ) noexcept;
```

If an `int` were to be returned from the `std::simd` overload of such functions then the size of the elements could change. For example, computing the bit width of a 8-bit integer could generate a simd of 64-bit integers as the output, which would lead to a dramatic change in storage size and performance. Instead, we propose that all the overloads for `<bit>` should return element types which are the same physical size as the element types they are querying. This would mean that calling `bit_width` on an unsigned 8-bit integer will return a SIMD value containing signed 8-bit values.

§ 3. Wording

Below, substitute the  character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 3.1. Add new section [simd.bit]

◆ basic_simd bit library [simd.bit]

```
template<typename T, typename Abi>
constexpr basic_simd<T, Abi> byteswap(const basic_simd<T, Abi>& n) noexcept
```

```
template<typename T, typename Abi>
constexpr basic_simd<T, Abi>::mask_type has_single_bit(const basic_simd<T, Abi>& n) noexcept
```

```
template<typename T, typename Abi>
constexpr basic_simd<T, Abi> bit_ceil(const basic_simd<T, Abi>& n) noexcept
```

```
template<typename T, typename Abi>
constexpr basic_simd<T, Abi> bit_floor(const basic_simd<T, Abi>& n) noexcept
```

```
template<typename T, typename Abi>
constexpr simd<std::make_signed<T>, basic_simd<T, Abi>>
bit_width(const basic_simd<T, Abi>& n) noexcept;
```

```
template<typename T, typename Abi>
[[nodiscard]] constexpr basic_simd<T, Abi> rotl(const basic_simd<T, Abi>& n) noexcept
```

```
template<typename T, typename Abi>
[[nodiscard]] constexpr basic_simd<T, Abi> rotr(const basic_simd<T, Abi>& n) noexcept
```

```
template<typename T, typename Abi>
constexpr simd<std::make_signed<T>, basic_simd<T, Abi>>
countl_zero(const basic_simd<T, Abi>& n) noexcept;
```

```
template<typename T, typename Abi>
constexpr simd<std::make_signed<T>, basic_simd<T, Abi>>
countl_one(const basic_simd<T, Abi>& n) noexcept;
```

```
template<typename T, typename Abi>
constexpr simd<std::make_signed<T>, basic_simd<T, Abi>>
countr_zero(const basic_simd<T, Abi>& n) noexcept;
```

```
template<typename T, typename Abi>
constexpr simd<std::make_signed<T>, basic_simd<T, Abi>>
countr_one(const basic_simd<T, Abi>& n) noexcept;
```

```
template<typename T, typename Abi>
constexpr simd<std::make_signed<T>, basic_simd<T, Abi>>
popcount(const basic_simd<T, Abi>& n) noexcept;
```

Constraints:

- Any constraints from the equivalent scalar function from `<bit>` will be applied to the equivalent element-wise function in `simd`.

Returns:

- A `basic_simd` with the same width as the input, where the i^{th} element will be equal to the result of applying the same function to the i^{th} element of the input.
- The element size of the return value will always be the same physical size as that of each input element, even when the equivalent scalar function would return a larger type.

Remarks:

- The order in which the functions are applied to each element is unspecified.

§ References

§ Informative References

[P1928R4]

Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 19 May 2023.
URL: <https://wg21.link/p1928r4>